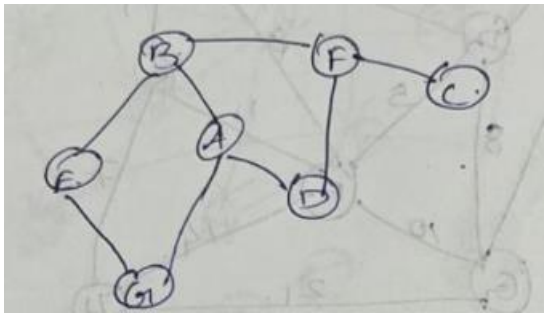
1)Simulate BFS and DFS traversals for the given graph.



CODE:

```
#include <stdio.h>
char vertex[V] = {'A', 'B', 'C', 'D', 'E', 'F', 'G'};
int graph[V][V] = {
    {0, 1, 0, 1, 0, 0, 0}, // A
    {1, 0, 0, 0, 1, 1, 0}, // B
    {0, 0, 0, 0, 0, 1, 0}, // C
    {1, 0, 0, 0, 0, 0, 1}, // D
    {0, 1, 0, 0, 0, 0, 1}, // E
    {0, 1, 1, 0, 0, 0, 1}, // F
    {0, 0, 0, 1, 1, 1, 0}  // G
};
int visited[V];
void BFS(int start)
{
    int queue[V], front = 0, rear = 0;
    int i, current;
    for (i = 0; i < V; i++)
        visited[i] = 0;
    queue[rear++] = start;
```

```
    visited[start] = 1;
    while (front < rear)
    {
        current = queue[front++];
        printf("%c ", vertex[current]);
        for (i = 0; i < V; i++)
        {
            if (graph[current][i] && !visited[i])
            {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
    }
}

void DFS(int current)
{
    int i;

    visited[current] = 1;
    printf("%c ", vertex[current]);

    for (i = 0; i < V; i++)
    {
        if (graph[current][i] && !visited[i])
            DFS(i);
    }
}
```

```
int main()
{
    int i;

    printf("BFS Traversal: ");
    BFS(0);

    for (i = 0; i < V; i++)
        visited[i] = 0;

    printf("\nDFS Traversal: ");
    DFS(0);

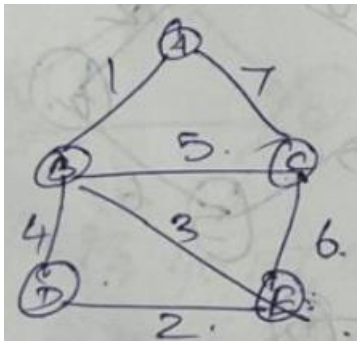
    printf("\n");
    return 0;
}
```

OUTPUT:

main.c | Output

```
BFS Traversal: A B D E F G C
DFS Traversal: A B E G D F C
```

2)Find out the minimum spanning tree for the given using prim's and kruskal's algorithms.



CODE:

```
#include <stdio.h>
#define MAX 100
int parent[MAX];
int find(int x)
{
    while (parent[x] != x)
        x = parent[x];
    return x;
}

int main()
{
    int n, e, i, j, u, v, w;
    int edgeU[MAX], edgeV[MAX], edgeW[MAX];
    int cost[MAX][MAX];
    int selected[MAX] = {0};
    int total, count, min, a, b, temp;

    printf("Enter number of vertices and edges: ");
    scanf("%d%d", &n, &e);

    for (i = 0; i < n; i++)
```

```
    for (j = 0; j < n; j++)
        cost[i][j] = (i == j) ? 0 : INF;

printf("Enter edges (u v weight), vertices numbered 0 to %d:\n", n - 1);

for (i = 0; i < e; i++)
{
    scanf("%d%d%d", &u, &v, &w);
    edgeU[i] = u;
    edgeV[i] = v;
    edgeW[i] = w;
    cost[u][v] = cost[v][u] = w;
}

/* Prim's Algorithm */
selected[0] = 1;
total = 0;
count = 0;

printf("\nPrim's MST:\n");

while (count < n - 1)
{
    min = INF;
    a = -1;
    b = -1;

    for (i = 0; i < n; i++)
    {
```

```
        if (selected[i])
        {
            for (j = 0; j < n; j++)
            {
                if (!selected[j] && cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = i;
                    b = j;
                }
            }
        }
    }

    if (a == -1)
    {
        printf("Graph is disconnected.\n");
        break;
    }

    printf("%d - %d : %d\n", a, b, min);
    selected[b] = 1;
    total += min;
    count++;
}

if (count == n - 1)
    printf("Total cost = %d\n", total);
```

```
/* Kruskal's Algorithm */
for (i = 0; i < e; i++)
   for (j = i + 1; j < e; j++)
      if (edgeW[i] > edgeW[j])
      {
         temp = edgeW[i];
         edgeW[i] = edgeW[j];
         edgeW[j] = temp;

         temp = edgeU[i];
         edgeU[i] = edgeU[j];
         edgeU[j] = temp;

         temp = edgeV[i];
         edgeV[i] = edgeV[j];
         edgeV[j] = temp;
      }

for (i = 0; i < n; i++)
   parent[i] = i;

total = 0;
count = 0;

printf("\nKruskal's MST:\n");

for (i = 0; i < e && count < n - 1; i++)
{
   u = find(edgeU[i]);
```

```
        v = find(edgeV[i]);

        if (u != v)
        {
            printf("%d - %d : %d\n",
                  edgeU[i], edgeV[i], edgeW[i]);

            parent[u] = v;
            total += edgeW[i];
            count++;
        }
    }

    if (count == n - 1)
        printf("Total cost = %d\n", total);
    else
        printf("Graph is disconnected.\n");

    return 0;
}
```

OUTPUT:

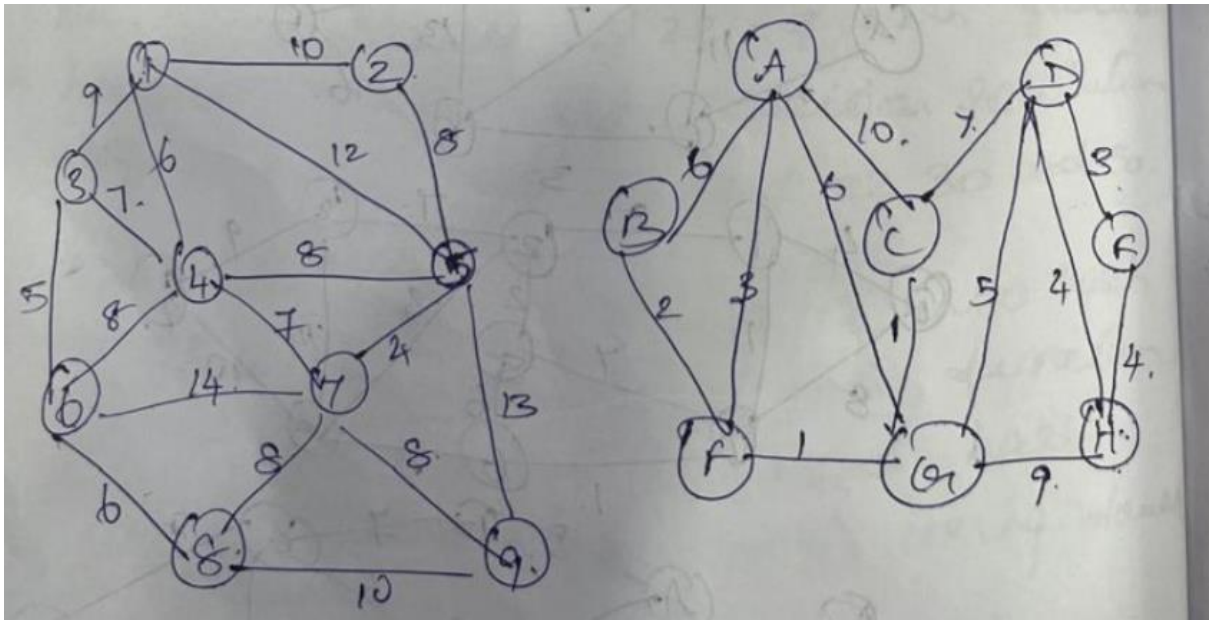
main.c | Output

```
Enter number of vertices and edges: 5 7
Enter edges (u v weight), vertices numbered 0 to 4:
0 1 1
0 2 7
1 2 5
1 3 4
1 4 3
2 4 6
3 4 2

Prim's MST:
0 - 1 : 1
1 - 4 : 3
4 - 3 : 2
1 - 2 : 5
Total cost = 11

Kruskal's MST:
0 - 1 : 1
3 - 4 : 2
1 - 4 : 3
1 - 2 : 5
Total cost = 11
```

3)To find the minimum spanning tree.



CODE:

```
#include <stdio.h>
#define MAX 20
int parent[MAX];
int find(int x)
{
   while (parent[x] != x)
      x = parent[x];
   return x;
}
int main()
{
   int n, e, i, j, u, v, w;
   int cost[MAX][MAX];
   int eu[MAX], ev[MAX], ew[MAX];
   int visited[MAX] = {0};
   int min, a, b, total, count, temp;

   printf("Enter number of vertices and edges: ");
```

```
scanf("%d%d", &n, &e);

for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        cost[i][j] = (i == j) ? 0 : INF;

printf("Enter edges (u v weight), vertices numbered 0 to %d:\n", n - 1);

for (i = 0; i < e; i++)
{
    scanf("%d%d%d", &u, &v, &w);
    eu[i] = u;
    ev[i] = v;
    ew[i] = w;
    cost[u][v] = cost[v][u] = w;
}

/* Prim's Algorithm */
visited[0] = 1;
total = 0;
count = 0;

printf("\nPrim's MST:\n");

while (count < n - 1)
{
    min = INF;
    a = -1;
    b = -1;
```

```
    for (i = 0; i < n; i++)
        if (visited[i])
            for (j = 0; j < n; j++)
                if (!visited[j] && cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = i;
                    b = j;
                }

    if (a == -1)
    {
        printf("Graph is disconnected.\n");
        break;
    }

    printf("%d - %d : %d\n", a, b, min);
    visited[b] = 1;
    total += min;
    count++;
}

if (count == n - 1)
    printf("Total cost = %d\n", total);

/* Sort edges for Kruskal's Algorithm */
for (i = 0; i < e; i++)
    for (j = i + 1; j < e; j++)
```

```
        if (ew[i] > ew[j])
        {
            temp = ew[i]; ew[i] = ew[j]; ew[j] = temp;
            temp = eu[i]; eu[i] = eu[j]; eu[j] = temp;
            temp = ev[i]; ev[i] = ev[j]; ev[j] = temp;
        }

for (i = 0; i < n; i++)
    parent[i] = i;

total = 0;
count = 0;

printf("\nKruskal's MST:\n");

for (i = 0; i < e && count < n - 1; i++)
{
    u = find(eu[i]);
    v = find(ev[i]);

    if (u != v)
    {
        printf("%d - %d : %d\n", eu[i], ev[i], ew[i]);
        parent[u] = v;
        total += ew[i];
        count++;
    }
}
```

```
    if (count == n - 1)
        printf("Total cost = %d\n", total);
    else
        printf("Graph is disconnected.\n");

    return 0;
}
```

OUTPUT:

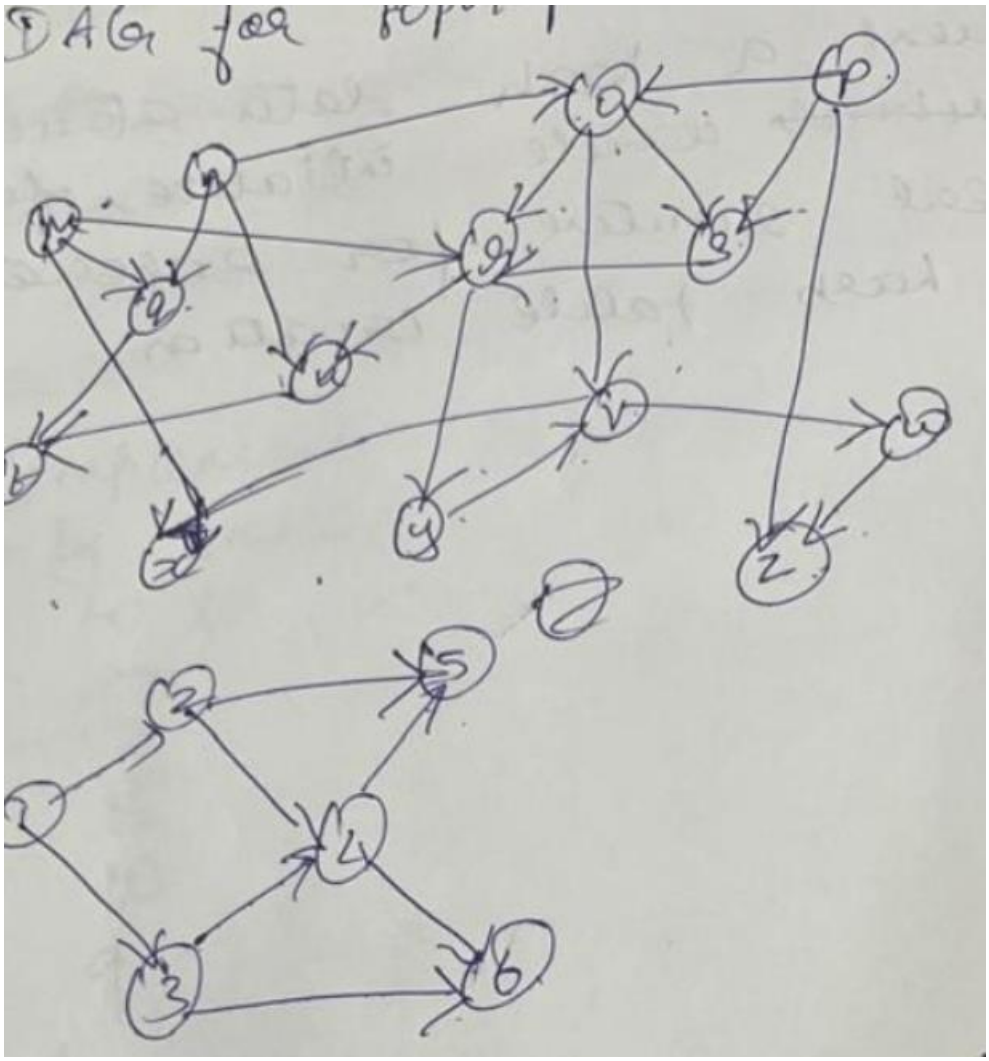
main.c | Output

```
0 6 6
2 6 1
2 3 7
5 6 1
6 3 5
3 4 3
3 7 4

Prim's MST:
0 - 5 : 3
5 - 6 : 1
6 - 2 : 1
5 - 1 : 2
6 - 3 : 5
3 - 4 : 3
3 - 7 : 4
Total cost = 19

Kruskal's MST:
2 - 6 : 1
5 - 6 : 1
1 - 5 : 2
0 - 5 : 3
3 - 4 : 3
3 - 7 : 4
6 - 3 : 5
Total cost = 19
```

4)DAG For topological sorting.



CODE:

```
#include <stdio.h>


int main()
{
    int n, i, j, count = 0;
    int a[20][20], indegree[20] = {0};
    int queue[20], front = 0, rear = -1;
    int topo[20];


    printf("Enter number of vertices: ");
```

```
scanf("%d", &n);

printf("Enter adjacency matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        scanf("%d", &a[i][j]);

for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        if (a[i][j] == 1)
            indegree[j]++;

for (i = 0; i < n; i++)
    if (indegree[i] == 0)
        queue[++rear] = i;

while (front <= rear)
{
    int u = queue[front++];
    topo[count++] = u;

    for (j = 0; j < n; j++)
    {
        if (a[u][j] == 1)
        {
            indegree[j]--;
            if (indegree[j] == 0)
                queue[++rear] = j;
        }
```

```
        }
    }

    if (count != n)
        printf("Graph contains a cycle.\n");
    else
    {
        printf("Topological order: ");
        for (i = 0; i < n; i++)
            printf("%d ", topo[i]);
        printf("\n");
    }

    return 0;
}
```

OUTPUT:

main.c | Output

```
Enter number of vertices: 6
Enter adjacency matrix:
0 0 0 0 0 0
0 0 0 0 0 0
0 1 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
Topological order: 0 2 3 4 5 1
```

5)Insert 76,93,40,47,10,55 in a hash table of size 7 using linear probing method.

CODE:

```
#include <stdio.h>
#define SIZE 7
int main()
{
    int hash[SIZE];
    int keys[] = {76, 93, 40, 47, 10, 55};
    int i, j, index;

    for (i = 0; i < SIZE; i++)
        hash[i] = -1;

    for (i = 0; i < 6; i++)
    {
```

```
        index = keys[i] % SIZE;

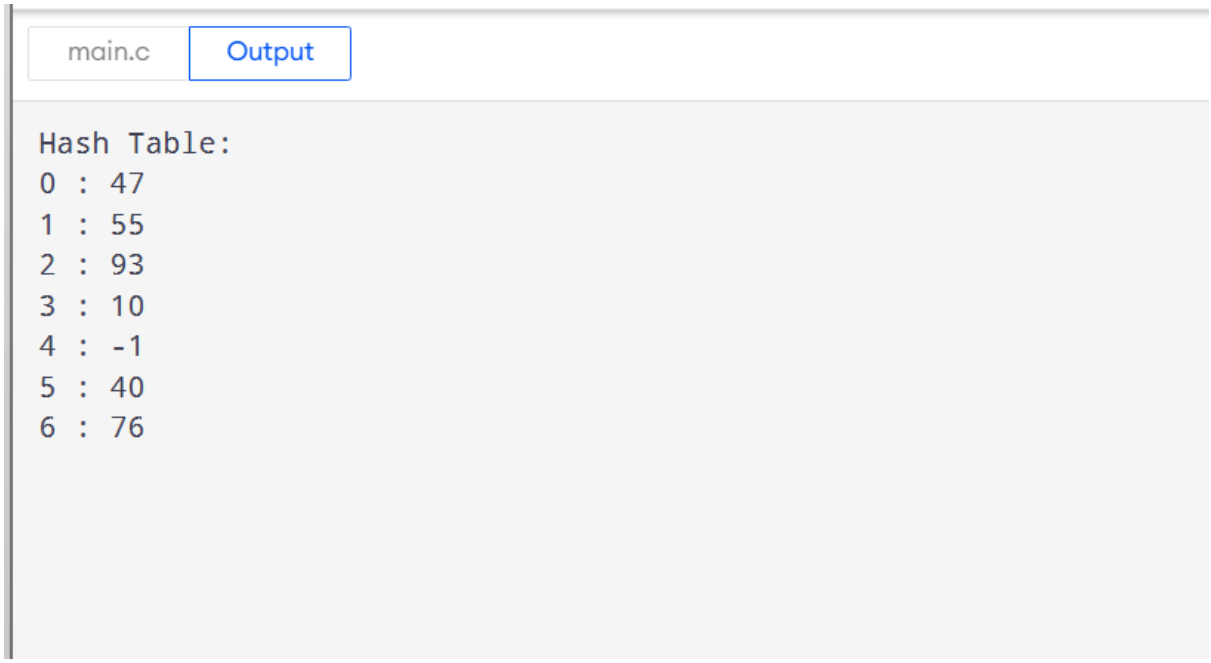
        for (j = 0; j < SIZE; j++)
        {
            if (hash[index] == -1)
            {
                hash[index] = keys[i];
                break;
            }

            index = (index + 1) % SIZE;
        }
    }

    printf("Hash Table:\n");
    for (i = 0; i < SIZE; i++)
        printf("%d : %d\n", i, hash[i]);

    return 0;
}
```

OUTPUT:



6)Consider a hash table size= 7, hash function as hash(x) = x%7 and collision resolution strategy to be f(i) = i^2 . Insert 22,30 and 50.

CODE:

```
#include <stdio.h>

#define SIZE 7

int main()

{

   int hash[SIZE];

   int keys[] = {22, 30, 50};

   int i, j, index;


   for (i = 0; i < SIZE; i++)

      hash[i] = -1;


   for (i = 0; i < 3; i++)

   {

      index = keys[i] % SIZE;


      for (j = 0; j < SIZE; j++)
```

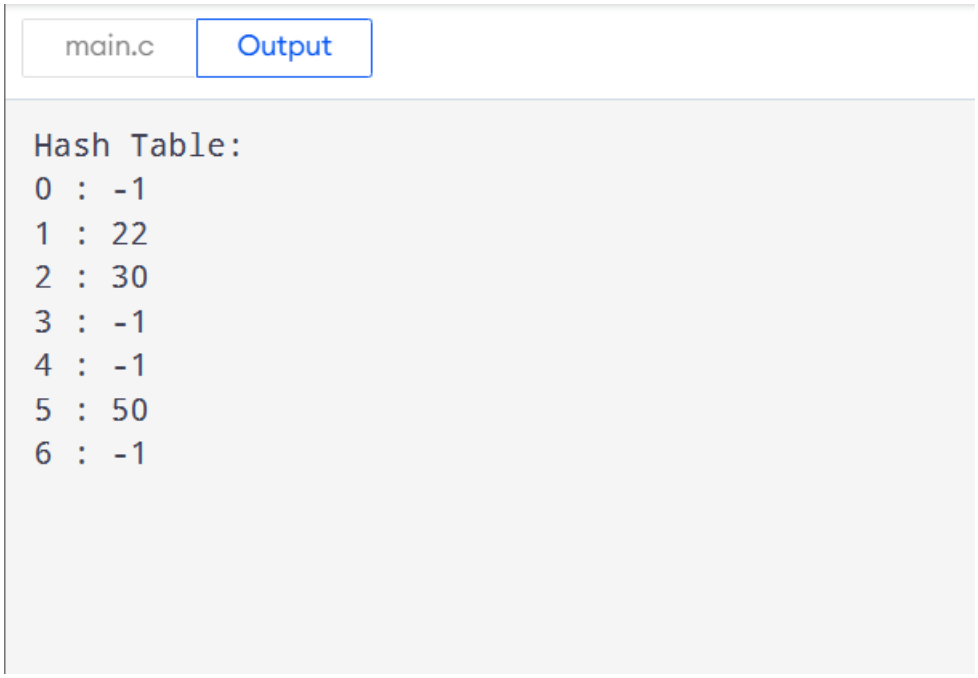
```
        {
            index = (keys[i] % SIZE + j * j) % SIZE;

            if (hash[index] == -1)
            {
                hash[index] = keys[i];
                break;
            }
        }
    }

    printf("Hash Table:\n");
    for (i = 0; i < SIZE; i++)
        printf("%d : %d\n", i, hash[i]);

    return 0;
}
```

OUTPUT:



7)Insert the keys 27,43,692,72 into hash table of size 7 where first hash function is h1(k)= k mod 7 and second hash function is h2(k)= 1+(k mod 5) using double hashing technique.

CODE:

```
#include <stdio.h>


#define SIZE 7


int main()
{
   int hash[SIZE];
   int keys[] = {27, 43, 69, 2, 72};
   int i, j, index, h1, h2;


   for (i = 0; i < SIZE; i++)
      hash[i] = -1;


   for (i = 0; i < 5; i++)
   {
      h1 = keys[i] % SIZE;
      h2 = 1 + (keys[i] % 5);


      for (j = 0; j < SIZE; j++)
      {
         index = (h1 + j * h2) % SIZE;


         if (hash[index] == -1)
         {
            hash[index] = keys[i];
            break;
```

```
        }
    }
}

printf("Hash Table:\n");
for (i = 0; i < SIZE; i++)
    printf("%d : %d\n", i, hash[i]);

return 0;
}
```

OUTPUT:

main.c | Output

```
Hash Table:
0 : -1
1 : 43
2 : 2
3 : -1
4 : 69
5 : 72
6 : 27
```

8)“If quadratic probing is used and the table size is prime, then a new element can always be inserted if the table is at least half empty.”

Proof

Let the hash table size be \(m\), where \(m\) is a prime number.

In quadratic probing, the positions checked are

h(x), h(x)+1^2, h(x)+2^2, h(x)+3^2,…….

All positions are calculated modulo \(m\).

1. Since the table size m is prime, the first (m+1)/2 probe positions are distinct.

2. Therefore, quadratic probing can check at least half of the table's positions without repeating any position.

3. Since the table is at least half empty, at least one of these positions must be empty.

4. Hence, the new element can be inserted into that empty position.

Hence proved.

In simple words: Quadratic probing checks different positions. If at least half the table is empty, one of the positions it checks must be empty, so insertion is possible.

9) Implement a hash data structure and write initialize , find and insert routine for separate chaining hash table creation

CODE:

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

struct node
{
   int data;
   struct node *next;
};

struct node *hashTable[SIZE];

/* Initialize hash table */
void initialize()
{
   int i;
```

```
    for (i = 0; i < SIZE; i++)
        hashTable[i] = NULL;
}

/* Hash function */
int hash(int key)
{
    return key % SIZE;
}

/* Find an element */
struct node *find(int key)
{
    int index = hash(key);
    struct node *temp = hashTable[index];

    while (temp != NULL)
    {
        if (temp->data == key)
            return temp;

        temp = temp->next;
    }

    return NULL;
}
void insert(int key)
{
```

```
    int index = hash(key);
    struct node *newNode;

    if (find(key) != NULL)
    {
        printf("Element already exists\n");
        return;
    }

    newNode = (struct node *)malloc(sizeof(struct node));
    if (newNode == NULL)
    {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = key;
    newNode->next = hashTable[index];
    hashTable[index] = newNode;

    printf("Element inserted successfully\n");
}
void display()
{
    int i;
    struct node *temp;

    for (i = 0; i < SIZE; i++)
    {
        printf("%d: ", i);
```

```
        temp = hashTable[i];

        while (temp != NULL)
        {
            printf("%d -> ", temp->data);
            temp = temp->next;
        }

        printf("NULL\n");
    }
}

int main()
{
    int choice, key;

    initialize();

    while (1)
    {
        printf("\n1. Insert\n2. Find\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter element: ");
                scanf("%d", &key);
```

```
                insert(key);
                break;

            case 2:
                printf("Enter element to find: ");
                scanf("%d", &key);

                if (find(key) != NULL)
                    printf("Element found\n");
                else
                    printf("Element not found\n");
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

OUTPUT:

```
1. Insert
2. Find
3. Display
4. Exit
Enter choice: 1
Enter element: 15
Element inserted successfully
Enter choice: 1
Enter element: 25
Element inserted successfully
Enter choice: 3
0: NULL
1: NULL
2: NULL
3: NULL
4: NULL
5: 25 -> 15 -> NULL
6: NULL
7: NULL
8: NULL
9: NULL
Enter choice:
```